MBLP131 · HAFTA 8

İşletim Sistemleri Temelleri

Hafta 8 · Ara Sınav Tekrarı ve Alıştırma

Yedi haftalık kuram, tek sınav: neyi bilmeli ve bunu nasıl puana çevirmeli?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

ŞİMDİYE KADAR

Neredeyiz: Dersin Yarısındayız

H1	H2	H3	H4	H5	H6	H7
Tarihçe	**Temeller**	**Süreçler**	**İş parçacıkları**	**Zamanlama, eşzamanlama**	**Bellek**	**SB ve dosyalar**
toplu işleme, çoklu programlama, zaman paylaşımı	hedefler, kesmeler, ikili mod, yapılar	sistem çağrıları, durumlar, PCB, fork()	modeller, Amdahl, zamanlama ölçütleri	FCFS … RR, muteks, semaforlar	MMU, parçalanma, sayfalama, TLB	sayfa hataları, FIFO/OPT/LRU, dizinler

Kuram: 1–7. haftalar — hepsi vizede

H8 Vize sınavı
bugün: tekrar ve alıştırma

Buradasınız ↑

H9–H14 Uygulamalı Linux
kabuk, dosyalar, izinler, borular, süreçler, betikler

H15 Final sınavı
dersin tamamı

Bugünün Gündemi

1. **Sınav** Pratik bilgiler, kapsam, soru türleri ve puanlar.
2. **Tekrar: 1–7. haftalar** Her hafta için bir iki slayt: tanımlar, diyagramlar, formüller.
3. **Formül kâğıdı** Dersin bütün formülleri tek slaytta.
4. **Çözümlü problemler** Yedi problem: sınavdaki her sayısal soru türü.
5. **Hatalar ve strateji** Puanlar genellikle nerede kaybedilir, 90 dakika nasıl kullanılır?
6. **Kısa sınav ve sorular** Kendinizi sınayın, sonra sınavla ilgili her şeyi sorun.

SINAV

Vize: Pratik Bilgiler

Ne zaman, nerede?

- Tarih ve saat: [duyurulacak]
- Salon: [duyurulacak]
- Süre: [90] dakika
- 10 dakika erken gelin; öğrenci kimliğinizi getirin.

İzin verilen malzeme

- Kalem; basit bir hesap makinesi [kesinleşecek]
- Kitap kapalı: not yok, telefon yok [kesinleşecek]
- Formül kâğıdı verilecek mi: [evet / hayır — kesinleşecek]

Biçim (örnek kâğıttaki gibi)

- A Bölümü: 10 çoktan seçmeli soru × 2 puan
- B Bölümü: 4 kısa yanıtlı soru × 5 puan
- C Bölümü: 5 sayısal problem, 60 puan
- Toplam: 100 puan

[Köşeli parantez] içindeki her şey öğretim üyesi tarafından dersin Telegram kanalında kesinleştirilecek. Sınav gününden önce kanala bakın.

SINAV

Kapsam: 1–7. Haftalar

Hafta	Konular	Tipik sorular
H1	Tarihçe: toplu işleme, çoklu programlama, zaman paylaşımı, UNIX, Linux	Dönüm noktaları; U = t_çalışma/(t_çalışma + t_hazırlık), U(n) = 1 − p^n
H2	Tanımlar, bakış açıları, hedefler, kesmeler, bellek hiyerarşisi, ikili mod, yapılar	Açıklama / karşılaştırma; mod biti; monolitik ve mikroçekirdek
H3	Hizmetler, sistem çağrıları, süreç, durumlar, PCB, bağlam değişimi, fork()	Durum diyagramı; ek yük s/(q + s); fork() ile süreç sayma
H4	İş parçacıkları, çoklu iş parçacığı modelleri, Amdahl yasası, zamanlama ölçütleri	Ortak ve özel öğeler; hızlanma; dönüş, bekleme, yanıt
H5	FCFS, SJF, SRTF, öncelikli, RR; yarış, kritik bölge, muteks, semafor	Gantt şeması + ortalama bekleme süresi; yarış sonuçları
H6	Bağlama, MMU, takas, uyan stratejileri, parçalanma, sayfalama, TLB, bölütleme	Yerleştirme; adres çevirisi; TLB ile EAT
H7	İsteğe bağlı sayfalama, sayfa hataları, FIFO/OPT/LRU, boşa çalışma, dosyalar, dizinler	Çerçeve tabloları; p ile EAT; blok hesabı; bağlantılar

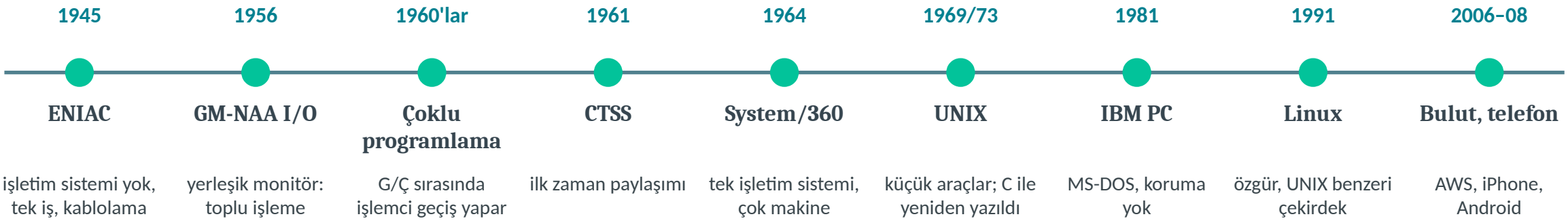
A ve B bölümleri her haftanın tanım ve açıklamalarını sorar; C bölümündeki sayısal problemler ağırlıklı olarak 5, 6 ve 7. haftalardan gelir.

BÖLÜM 1

Tekrar: 1–7. Haftalar

Bilmeniz gereken tanımlar, diyagramlar ve formüller — ders notlarındaki sözcüklerle.

İşletim Sistemleri Neden Var?



Her biri tek satırda

- **Toplu işleme:** yerleşik monitör işleri art arda çalıştırır; insanın hazırlık süresi ortadan kalkar.
- **Çoklu programlama:** bellekte birkaç iş; biri G/Ç beklerken işlemci başkasına geçer.
- **Zaman paylaşımı:** sırayla kısa zaman dilimleri; her kullanıcı makinenin tamamı kendisininmiş gibi hisseder.

```
# işlemci kullanım oranı, toplu işleme
U = t_çalışma / (t_çalışma + t_hazırlık)

# n iş, her biri p kesri kadar G/Ç bekler
U(n) = 1 − pⁿ

# zaman paylaşımı: en kötü durumda bekleme
T_bekleme ≈ n · q
```

İşletim Sistemi Nedir, Kendini Nasıl Korur?

Tanım ve bakış açıları

- Çekirdek: her zaman çalışan tek program.
- Genişletilmiş makine (yukarıdan aşağı): temiz soyutlamalar — dosyalar, süreçler.
- Kaynak yöneticisi (aşağıdan yukarı): zamanda ve uzayda çoğullama.

Hedefler ve bileşenler

- Hedefler: kullanım kolaylığı, verimlilik, gelişebilirlik — birbiriyle çatışırlar.
- Dört bileşen: donanım, işletim sistemi, uygulama programları, kullanıcılar.

Kesmeler

- Aygıt işlemciye sinyal verir; durum kaydedilir; işleyici kesme vektöründe bulunur; durum geri yüklenir.
- İşletim sistemi kesme güdümlüdür; yoklama işlemciyi boşa harcar.

Bellek hiyerarşisi

- Yazmaçlar ≈ 0,3 ns, önbellek, RAM ≈ 100 ns, SSD ≈ 50 µs, HDD ≈ 5 ms.
- Yazmaçlar, önbellek ve RAM uçucudur.

İkili mod

- Mod biti 0 = çekirdek modu, 1 = kullanıcı modu.
- Kullanıcı modunda ayrıcalıklı komut → tuzak.
- Sistem çağrısı: çekirdek moduna geçiş ve geri dönüş.

Sistem çağrısının maliyeti

- Ölçtük: sistem çağrısı başına ≈ 116 ns, fonksiyon çağrısı başına ≈ 1,5 ns.
- Kullanıcı/çekirdek sınırını geçmek pahalıdır.

Bir İşletim Sistemini Kurmanın Altı Yolu

Yapı	Çekirdek modunda	Hız	Güvenilirlik	Örnek
Basit	her şey (koruma yok)	yüksek	çok düşük	MS-DOS
Monolitik	tüm işletim sistemi hizmetleri	yüksek	orta	UNIX, Linux
Katmanlı	tüm katmanlar	düşük	yüksek	THE (1968)
Mikroçekirdek	yalnızca küçük çekirdek	daha düşük	yüksek	MINIX 3, QNX, seL4
Modüler	öz + yüklenen modüller	yüksek	orta	Linux, Solaris
Hibrit	değişir	yüksek	orta–yüksek	Windows, macOS, Android

Tek bir isteğin maliyeti: $T_{mono} = 2t_s + t_w$, buna karşılık $T_{\mu} = 4t_s + 2t_c + t_w$ (4 mod geçişi + 2 ileti kopyası). Her sınır güvenilirlik kazandırır, hız kaybettirir.

printf()'ten Çekirdeğe ve Geri



Hizmetler

- **Kullanıcı için:** kullanıcı arayüzü, program yürütme, G/Ç, dosya sistemi işlemleri, iletişim, hata algılama.
- **Sistem için:** kaynak ayırma, hesap tutma ve kayıt, koruma ve güvenlik.

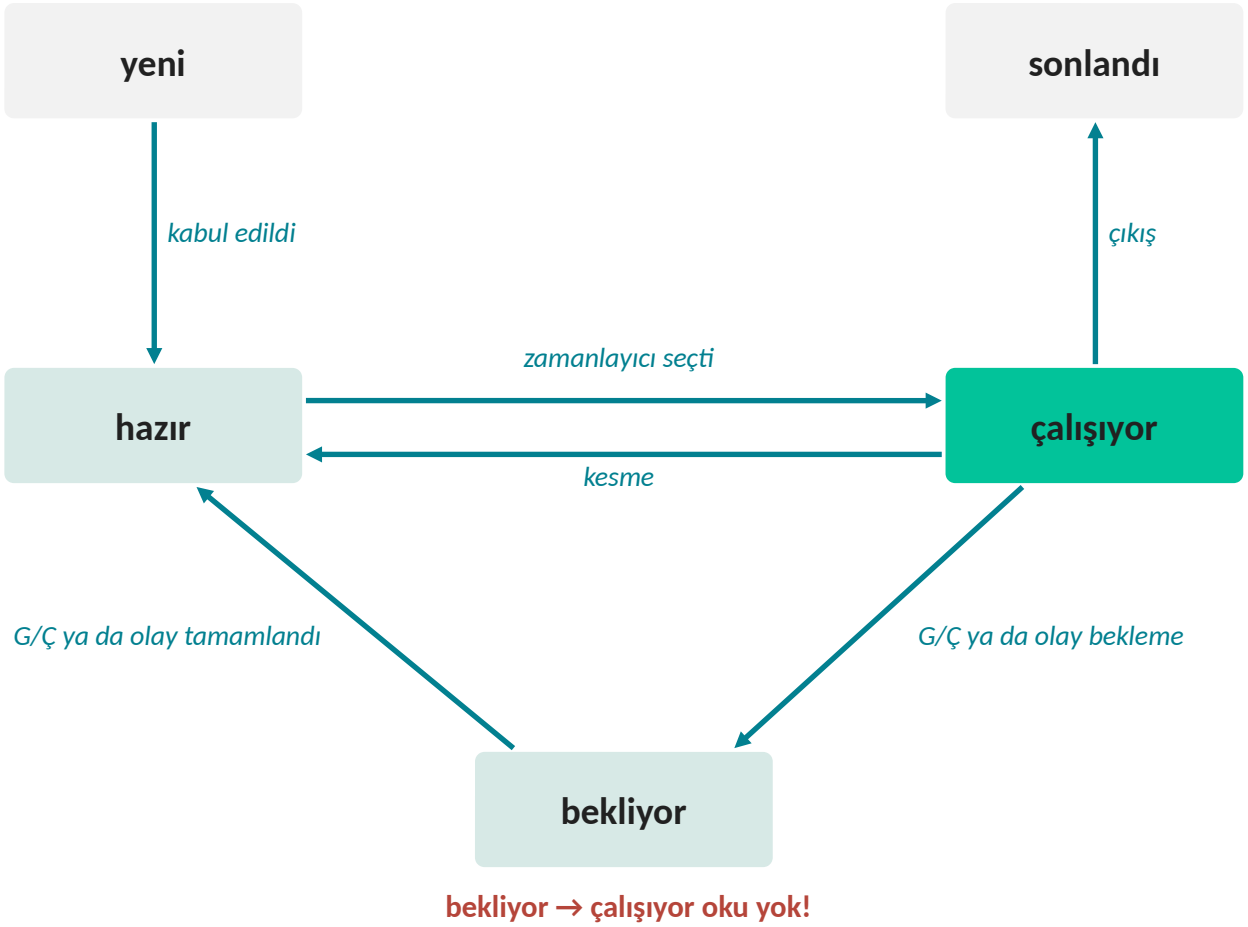
Parametre aktarımı

- 1. Yazmaçlarda (en hızlı).
- 2. Bellekte bir blokta; adresi bir yazmaçta.
- 3. Yığıtta.
- x86-64 Linux: numara rax'ta, en çok altı parametre yazmaçlarda.

Altı tür

- Süreç denetimi (fork, exit, wait)
- Dosya yönetimi (open, read, close)
- Aygıt yönetimi (ioctl)
- Bilgi bakımı (getpid)
- İletişim (pipe, socket)
- Koruma (chmod)

Süreç Durumları, PCB, Bağlam Değişimi, fork()



PCB ve bağlam değişimi

- PCB: durum, PID, program sayacı, yazmaçlar, zamanlama, bellek ve hesap bilgileri, açık dosyalar.
- Bağlam değişimi = durumu eski PCB'ye kaydet, yeni PCB'yi yükle. Saf ek yük: s / (q + s).

Süreç oluşturma

- fork() çocuğa 0, ebeveyne çocuğun PID'sini döndürür.
- exec() programı değiştirir; wait() çocuğu toplar.
- Art arda n fork → 2^n süreç. Zombi: bitmiş ama henüz beklenmemiş.

İş Parçacıkları, Modeller, Amdahl Yasası

Tüm iş parçacıklarında ortak	Her iş parçacığına özel
kod (metin bölümü)	iş parçacığı kimliği
global veri ve yığın (heap)	program sayacı ve yazmaçlar
açık dosyalar, PID, kullanıcı kimliği	yığıt (yerel değişkenler)

İş parçacıklarının yararları

Tepki verebilirlik, kaynak paylaşımı, ekonomi (süreçten ucuz: test makinemizde ≈ 4 kat), çok çekirdekli işlemcilerde ölçeklenebilirlik.

Modeller (kullanıcı : çekirdek iş parçacığı)

- **Çoktan bire:** hızlı, ama tek bir bloklayan çağrı hepsini durdurur; paralellik yok.
- **Birebir:** Linux ve Windows; gerçek paralellik; her iş parçacığı bir çekirdek iş parçacığına mal olur.
- **Çoktan çoğa:** Go goroutine'leri, Java sanal iş parçacıkları.
- Eşzamanlılık = birlikte ilerleme; paralellik = aynı anda (≥ 2 çekirdek gerekir).

```
# Amdahl yasası: seri kesir S, N çekirdek
hızlanma(N) ≤ 1 / (S + (1 − S)/N)
sınır N → ∞:  1 / S

# S = 0.25:
N = 2 → 1.60   N = 4 → 2.29
N = 8 → 2.91   N → ∞ → 4
```

Patlamalar, Karar Noktaları, Dağıtıcı, Ölçütler

CPU–G/Ç patlama döngüsü

İşlemci yoğun: az sayıda uzun CPU patlaması. G/Ç yoğun: çok sayıda kısa patlama. Patlamaların çoğu kısadır.

Dört karar noktası

- 1 çalışıyor → bekliyor 2 çalışıyor → hazır
- 3 bekliyor → hazır 4 sonlanma
- Kesintisiz: yalnızca 1 ve 4. Kesintili: dördü de.

Dağıtıcı (dispatcher)

Bağlamı değiştirir, kullanıcı moduna geçer, kaydedilmiş PC'ye atlar. Süresi = dağıtım gecikmesi.

```
T_dönüş   = T_tamamlanma – T_geliş
T_bekleme = T_dönüş – T_patlama
T_yanıt   = T_ilk çalışma – T_geliş

işlemci kullanımı = B / T  (meşgul / toplam)
üretilen iş       = n / T

en büyük: kullanım, üretilen iş
en küçük: dönüş, bekleme, yanıt
```

Bekleme süresi = yalnızca hazır kuyruğunda geçen süre. Zamanlayıcı patlamaları ya da G/Ç'yi değiştiremez — yalnızca beklemeyi.

Beş Algoritmaya Toplu Bakış

Algoritma	Kural	Kesintili mi?	Dikkat
FCFS	FIFO hazır kuyruğu, bitene kadar çalışır	hayır	konvoy etkisi: kısa işler uzun bir işin arkasında
SJF	en kısa sonraki CPU patlaması önce; $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$	hayır	en iyi ortalama bekleme (hepsi hazırsa); tahmin gerekir
SRTF	KALAN süreden kısa olan yeni gelen işlemciyi alır	evet	en küçük ortalama bekleme; uzun işler aç kalabilir
Öncelikli	en yüksek öncelik önce (küçük sayı = yüksek)	ikisi de	açlık → çaresi: yaşlandırma
Döngüsel (RR)	zaman dilimi q, kesilen süreç kuyruğun sonuna	evet	q çok büyük → FCFS; çok küçük → ek yük s/(q + s)

Sınav uzlaşımları (yazın): bağlam değişimi = 0 · eşitlikte önce erken gelen, sonra ad · küçük sayı = yüksek öncelik · RR: zaman dilimi biterken gelen süreç kuyruğa kesilenden ÖNCE girer.

Yarış Durumu, Kritik Bölge, Semaforlar

Yarış durumu

- Birkaç iş parçacığı ortak veriye erişir, en az biri yazar, sonuç zamanlamaya bağlıdır.
- counter++ = oku – ekle – yaz: bir güncelleme kaybolabilir.

Kritik bölge problemi

- Karşılıklı dışlama: içeride en çok bir iş parçacığı.
- İlerleme: dışarıdan veto yok, sonsuz erteleme yok.
- Sınırlı bekleme: açlık yok.

Kilitler

- Kesmeleri kapatmak: çok işlemcide olmaz, ayrıcalıklı.
- Peterson (2 iş parçacığı); atomik test_and_set / CAS.
- Muteks: acquire / release; döner kilit ya da uyuyarak bekleme.

Semaforlar

- wait(S): while (S <= 0); S--; signal(S): S++;
- İkili (= muteks, başlangıç 1) ya da sayan (başlangıç N).
- Sınırlı tampon: empty = n, full = 0, mutex = 1.

Kilitlenme: bir gruptaki her iş parçacığı yalnızca grubun başka bir üyesinin yol açabileceği bir olayı bekler (örn. wait(S); wait(Q) ile wait(Q); wait(S)). Önlem: kilitleri tek bir genel sırayla alın.

Bağlama, MMU, Yer Ayırma ve Parçalanma

Adres bağlama

- Derleme zamanı: mutlak kod.
- Yükleme zamanı: yeri değiştirilebilir kod.
- Yürütme zamanı: MMU her erişimi çevirir — tüm modern sistemler.

Taban ve sınır

- Mantıksal a: 0 ≤ a < sınır ise fiziksel = taban + a; değilse tuzak.
- Bu yazmaçları yalnızca çekirdek yükleyebilir.

Takas

- Süre = boyut / aktarım hızı (100 MB, 50 MB/s: her yön 2 s).
- Modern sistemler süreçleri değil sayfaları takas eder.

Yerleştirme

- İlk uyan: yeterince büyük ilk delik.
- En iyi uyan: yeterince büyük en küçük delik.
- En kötü uyan: en büyük delik.

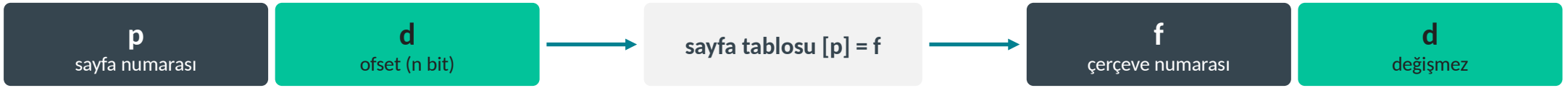
Parçalanma

- Dış: boş bellek, kullanılamayacak kadar küçük deliklerde (%50 kuralı: 1/3'e kadar kayıp).
- İç: ayrılmış bir bloğun içindeki kullanılmayan bellek.

Sıkıştırma

- Blokları bir araya taşı, tek delik kalsın.
- Yürütme zamanında bağlama gerekir; pahalıdır.
- Daha iyi çare: sayfalama.

Sayfalar, Çerçeveler, TLB ve Bölütler




```
sayfa boyu 2ⁿ:  p = ⌊a / 2ⁿ⌋,   d = a mod 2ⁿ
a_fiz = f · 2ⁿ + d
4 KB sayfa + onaltılık: sayfa basamağını değiştir
tablo boyu = (2ᵐ / 2ⁿ) girdi × girdi boyu
32 bit, 4 KB, 4 B → 2²⁰ girdi = 4 MB
```

```
# ders uzlaşımı (TLB araması hep sayılır)
EAT = α(t_TLB + t_m) + (1 − α)(t_TLB + 2t_m)
    = t_TLB + (2 − α) t_m
# sadeleştirilmiş (t_TLB ≈ 0):
EAT = α t_m + (1 − α) 2t_m
```

Sayfalama

Dış parçalanma yok; iç: süreç başına ortalama yarım sayfa. Geçerli–geçersiz bit ve r/w/x bitleri her erişimde denetlenir.

Bölütleme

Adres ⟨s, d⟩: 0 ≤ d < sınır[s] ise taban[s] + d, değilse tuzak. Değişken boy → dış parçalanma. x86-64 Linux: düz bölütler, işi sayfalama yapar.

İsteğe Bağlı Sayfalama, Sayfa Hataları, Değiştirme

1 Başvuru bit = i → **2 Tuzak** MMU → işletim sistemi → **3 Bulma** diskteki sayfa, boş çerçeve → **4 Okuma** disk → çerçeve → **5 Güncelleme** çerçeve no., bit = v → **6 Yeniden başlatma** aynı komut

```
EAT = (1 − p) t_m + p t_fault
p = sayfa hatası oranı; t_fault ≈ ms
→ p çok küçük olmalı (≈ 10⁻⁶)
```

Algoritma	Kurban	Not
FIFO	en önce yüklenen	ucuz; Belady anomalisi olabilir
OPT	en uzak gelecekte kullanılacak	en az hata; geleceği ister
LRU	en uzun süredir kullanılmayan	iyi; saat algoritmasıyla yaklaşılır

Bunları da bilin

- Geçerli–geçersiz bit; saf isteğe bağlı sayfalama; sayfalayıcı.
- Yazınca kopyala: fork() sayfaları ilk yazmaya kadar paylaşır.
- Değişiklik (kirli) biti: temiz kurban diske geri yazılmaz.

Boşa çalışma ve çalışma kümesi

- Çok az çerçeve → sürekli hata → işlemci boşta → daha çok süreç alınır → daha kötü.
- WS(t, Δ) = son Δ başvurudaki farklı sayfalar; $D = \Sigma WSS_i > m$ çerçeve ise → bir süreci askıya al.

Dosyalar, Erişim Yöntemleri, Dizinler, Bağlantılar

Dosya ve düğüm (inode)

- Öznitelikler: ad, kimlik, tür, konum, boyut, koruma, zaman damgaları.
- Linux: hepsi düğümde — ad hariç (o dizindedir).

İşlemler, tablolar

- create, open, read, write, seek, close, delete (unlink), truncate.
- Bir kez open() → dosya tanımlayıcısı; süreç başına tablo (dosya işaretçisi), sistem geneli tablo (açılma sayısı).

```
# doğrudan erişim, k. kayıt
konum   = k · R
blok    = ⌊k·R / B⌋
blok içi = k·R − blok · B
SON baytı da denetle!
```

Erişim yöntemleri

- **Sıralı:** sırayla (teyp modeli).
- **Doğrudan:** herhangi bir n. blok (disk modeli).
- **İndeksli:** küçük bir indekste ara, sonra atla.

Dizin yapıları

- Tek düzeyli → iki düzeyli → ağaç → döngüsüz çizge → genel çizge (döngüler).
- Mutlak yol / ile başlar; göreli yol çalışma dizininden; . ve ..

Bağlantılar

- **Sabit:** aynı düğüm için başka bir ad; bağlantı sayısı.
- **Sembolik:** bir yol tutan dosya; hedef silinirse boşta kalır.

1–7. Haftaların Bütün Formülleri Tek Slaytta

Hafta	Büyüklük	Formül
1	Toplu işleme kullanımı / çoklu programlama	$U = t_{çalışma} / (t_{çalışma} + t_{hazırlık})$; $U(n) = 1 - p^n$
2	Mikroçekirdek ve monolitik (tek istek)	$T_{mono} = 2t_s + t_w$; $T_\mu = 4t_s + 2t_c + t_w$
3, 5	Bağlam değişimi (RR) ek yükü	$s / (q + s)$
3	Art arda n fork() çağrısından sonra süreç	2^n ($2^n - 1$ tanesi yeni)
4	Amdahl yasası	hızlanma $\le 1 / (S + (1 - S)/N)$; sınır $1/S$
4	Dönüş ve bekleme süresi	$T_{dönüş} = T_{tamamlanma} - T_{geliş}$; $T_{bekleme} = T_{dönüş} - T_{patlama}$
4	Yanıt süresi	$T_{yanıt} = T_{ilk çalışma} - T_{geliş}$
5	Patlamaların üstel ortalaması	$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$
6	Taban ve sınır	$0 \le a < sınır \rightarrow$ taban + a, değilse tuzak
6	Sayfalama	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$; $a_{fiz} = f \cdot 2^n + d$
6	TLB ile etkin erişim süresi	$EAT = t_{TLB} + (2 - \alpha)\, t_m$
7	Sayfa hatalı etkin erişim süresi	$EAT = (1 - p)\, t_m + p\, t_{fault}$
7	Doğrudan erişim, k. kayıt	blok = $\lfloor kR / B \rfloor$; blok içi konum = $kR - blok \cdot B$

ARA

10 Dakika Mola

Sırada: yedi çözümlü problem — her sayısal soru türü için bir tane.

BÖLÜM 2

Çözümlü Problemler

Önce her birini kendiniz deneyin. Her sayı code/review_problems.py ile denetlendi.

1. PROBLEM · 1, 3, 4. HAFTALAR

Dört Kısa Hesap

(a) Çoklu programlama (H1)

- İşler zamanın p = 0,6'sında G/Ç bekliyor. U(3)? U ≥ %95 için en küçük n?
- **U(3) = 1 − 0,6³ = %78,4.** $0{,}6^5 \approx 0{,}078 > 0{,}05$ ama $0{,}6^6 \approx 0{,}047 \le 0{,}05 \rightarrow n = 6$.

(b) Bağlam değişimi (H3)

- s = 4 µs. q = 2 ms ve q = 50 µs için ek yük?
- **4 / 2004 ≈ %0,20;** 4 / 54 ≈ %7,4. Çok küçük zaman dilimleri işlemciyi boşa harcar.

(c) fork() (H3)

- fork(); fork(); printf("A\n"); fork(); printf("B\n");
- **A: 4 kez, B: 8 kez,** 8 süreç (7 tanesi yeni). Çıktı terminale gidiyor.

(d) Amdahl (H4)

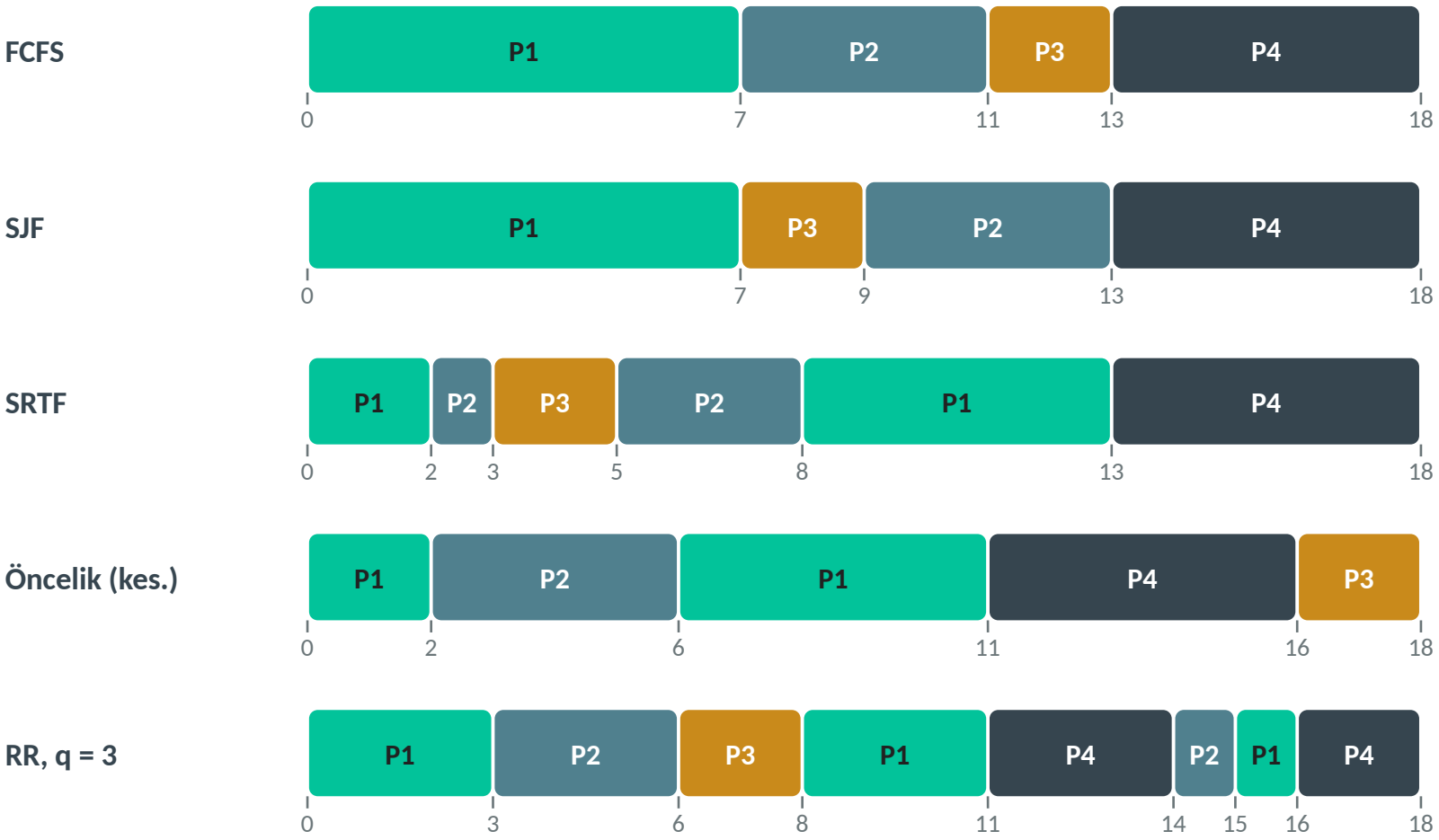
- S = 0,2. 4 ve 8 çekirdekte hızlanma? Sınır?
- **1/(0,2 + 0,8/4) = 2,5;** 1/(0,2 + 0,1) ≈ 3,33; sınır 1/0,2 = 5.

Tek İş Kümesi, Beş Zamanlayıcı

Süreç	Varış	Patlama	Öncelik
P1	0	7	2
P2	2	4	1
P3	3	2	4
P4	5	5	3

Şemayı okuyun

- SRTF t = 3: P3 (2) < P2'nin kalanı 3 → kesme.
- SRTF t = 8: P1 ve P4'ün ikisi de 5 ister → erken gelen P1.
- RR t = 3: P3, P1'in zaman dilimi biterken gelir → önce P3.



Sonuçlar: Ortalamalar ve Bağlam Değişimleri

Algoritma	Kesintili	Ort. bekleme	Ort. dönüş	Ort. yanıt	Geçiş
FCFS	hayır	5,25	9,75	5,25	3
SJF	hayır	4,75	9,25	4,75	3
SRTF	evet	4	8,5	2	5
Öncelikli	evet	5,75	10,25	4,75	4
RR, q = 3	evet	7,25	11,75	2,5	7

```
# SRTF, formüllerden
P1: dönüş 13 − 0 = 13, bekleme 13 − 7 = 6
P2: dönüş  8 − 2 = 6, bekleme  6 − 4 = 2
P3: dönüş  5 − 3 = 2, bekleme  2 − 2 = 0
P4: dönüş 18 − 5 = 13, bekleme 13 − 5 = 8
ortalama bekleme = 16/4 = 4 ms
```

Sayılar ne söylüyor?

- SRTF en küçük ortalama bekleme süresini (4 ms) verir — kuramın söylediği gibi — ama RR'den sonra en çok bağlam değişimini yapar.
- Öncelikli zamanlama ortalamaya değil öneme hizmet eder: düşük öncelikli P3 13 ms bekler. RR en kötü ortalama beklemeyi (7,25 ms) ama iyi bir yanıt süresini (2,5 ms) verir.

Bir Yarış ve Bir Semafor, Adım Adım

Adım	A: counter++	B: counter--	counter
1	oku (rax = 10)		10
2		oku (rax = 10)	10
3	1 ekle (rax = 11)		10
4		−1 ekle (rax = 9)	10
5	yaz		11
6		yaz	9 (10 olmalı)

Olay	S	Sonuç
başlangıç (2 yazıcı)	2	
T1 wait	1	T1 girer
T2 wait	0	T2 girer
T3 wait	0	T3 bloklanır
T1 signal	1 → 0	T3 girer
T2 signal	1	
T3 signal	2	hepsi bitti

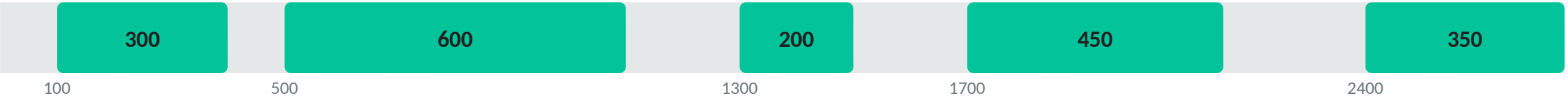
Altı adımın 20 iç içe geçmesinin hepsi denetlendi: counter 9 (9 sıra), 10 (2 sıra) ya da 11 (9 sıra) ile biter. 20 sıranın yalnızca 2'si doğru!

Sayan semafor S = 2: aynı anda en çok iki iş parçacığı yazıcı kullanır.

4. PROBLEM · 6. HAFTA

İlk Uyan, En İyi Uyan, En Kötü Uyan

Delikler (nane, boyut KB) adres sırasında; gri = kullanımda. İstekler sırayla: A = 340, B = 180, C = 420, D = 500 KB.



İstek	İlk uyan	En iyi uyan	En kötü uyan
A = 340	600@500 → kalan 260	350@2400 → kalan 10	600@500 → kalan 260
B = 180	300@100 → kalan 120	200@1300 → kalan 20	450@1700 → kalan 270
C = 420	450@1700 → kalan 30	450@1700 → kalan 30	beklemek zorunda
D = 500	beklemek zorunda	600@500 → kalan 100	beklemek zorunda

İlk uyan: 960 KB boş olduğu hâlde D bekler (en büyük delik 350 KB) — dış parçalanma. En iyi uyan dördünü de yerleştirir. En kötü uyan iki kez başarısız: 1380 KB boş, en büyük delik 350 KB. Tek bir örnek genel bir şey kanıtlamaz.

Adres Çevirisi ve TLB

Sayfa	Çerçeve	Geçerli
0	3	v
1	8	v
2	6	v
3	—	i
4	10 (0xA)	v
5	2	v

16 bitlik adresler, 4 KB sayfalar: p = ilk onaltılık basamak, d = son üç basamak.

```
0x1C3B: p = 1, d = 0xC3B
  çerçeve 8 → 0x8C3B = 35899
  (8 × 4096 + 3131)

0x4F0E: p = 4, d = 0xF0E
  çerçeve 10 → 0xAF0E = 44814

0x2010: p = 2 → 0x6010 = 24592

0x3004: p = 3 geçersiz
  → TUZAK (çeviri yok)
```

```
t_TLB = 10, t_m = 80 ns, α = 0.95
EAT = 0.95 × 90 + 0.05 × 170
    = 85.5 + 8.5 = 94 ns
denetim: 10 + 1.05 × 80 = 94
TLB'siz: 2 × 80 = 160 ns

EAT = 100 ns → α = ?
10 + (2 − α) 80 = 100
α = 0.875
```

Sadeleştirilmiş (t_TLB ≈ 0)

0,95 × 80 + 0,05 × 160 = 84 ns. Uzlaşımınızı yazın!

Sayfa Değiştirme: FIFO ve LRU, 3 Çerçeve

Başvuru dizisi: 1, 2, 0, 4, 2, 3, 2, 1, 2, 4, 0, 2, 0, 1 · 3 çerçeve, başlangıçta boş

FIFO

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	**1**	1	1	**4**	4	4	4	**1**	1	1	1	**2**	2	2
çerçeve 1		**2**	2	2	2	**3**	3	3	3	**4**	4	4	4	**1**
çerçeve 2			**0**	0	0	0	**2**	2	2	2	**0**	0	0	0
hata?	F	F	F	F		F	F	F		F	F	F		F

11
FIFO ile hata

LRU

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	**1**	1	1	**4**	4	4	4	**1**	1	1	**0**	0	0	0
çerçeve 1		**2**	2	2	2	2	2	2	2	2	2	2	2	2
çerçeve 2			**0**	0	0	**3**	3	3	3	**4**	4	4	4	**1**
hata?	F	F	F	F		F		F		F	F			F

9
LRU ile hata

Sayfa Değiştirme: OPT ve Karşılaştırma

OPT (ileriye bak: en uzak gelecekte kullanılacak sayfayı değiştir)

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
çerçeve 1		2	2	2	2	2	2	2	2	2	2	2	2	2
çerçeve 2			0	4	4	3	3	3	3	4	0	0	0	0
hata?	F	F	F	F		F				F	F			

7
OPT ile hata (en az)



14 başvuru içinde

Yapabileceğiniz denetimler

- Zorunlu hatalar: 5 farklı sayfa (0, 1, 2, 3, 4) → her algoritmada en az 5 hata.
- Her zaman OPT ≤ LRU ve OPT ≤ FIFO.
- 4 çerçeveyle: FIFO 8, LRU 7, OPT 6 — burada anomali yok.

Sayfa Hatalı EAT ve Doğrudan Erişim Hesabı

(a) t_m = 100 ns, t_fault = 6 ms, p = 0,0002

```
t_fault = 6 ms = 6,000,000 ns
EAT = 0.9998 × 100 + 0.0002 × 6,000,000
    = 99.98 + 1,200 = 1299.98 ns
→ RAM'den yaklaşık 13 kat yavaş

yavaşlama < %5:  EAT < 105 ns
100 + p (6,000,000 − 100) < 105
p < 5 / 5,999,900 ≈ 8.3 × 10⁻⁷
```

(b) 120 baytlık kayıtlar, 4096 baytlık bloklar: 750. kayıt?

```
konum    = 750 × 120 = 90,000
blok     = ⌊90,000 / 4096⌋ = 21
blok içi = 90,000 − 21 × 4096 = 3984

son bayt = 90,000 + 119 = 90,119
⌊90,119 / 4096⌋ = 22  → sonraki blok!
→ 21. bloğun 3984…4095 baytları
  ve 22. bloğun 0…7 baytları: iki okuma
```

Ders (a): önce her şeyi ns'ye çevirin; 5.000 erişimde tek bir hata bile belleği 13 kat yavaşlatır. Ders (b): kaydın son baytını her zaman denetleyin — burada kayıt bir blok sınırını aşıyor.

SINAV TEKNİĞİ

Puanlar Genellikle Nerede Kaybedilir?

Zamanlama

- Bekleme = dönüş − patlama, 'başlama zamanı' değil.
- SRTF KALAN süreyle karşılaştırır.
- RR: yeni gelen kesilen süreçten önce.

Birimler

- 8 ms = 8.000.000 ns; 2 ms = 2000 µs.
- Sayfalamada KB = 1024 B; takas sürelerinde MB/s verildiği gibi.

Sayfalama

- Ofset hiçbir zaman çevrilmez.
- Geçersiz sayfa → tuzak, sayı değil.
- a = sınır zaten tuzak (katı <).

Çerçeve tabloları

- FIFO: isabet sırayı değiştirmez.
- LRU: isabet sayfayı en yeni yapar.
- Hataları iki kez sayın.

Tanımlar

- Eşzamanlılık ≠ paralellik.
- Karşılıklı dışlama ≠ ilerleme ≠ sınırlı bekleme.
- Açlık ≠ kilitlenme.

Yanıtlar

- Uzlaşımları ve birimleri yazın.
- Sayıdan önce formülü gösterin.
- 'Neden'i tek cümleyle açıklayın.

SINAV TEKNİĞİ

90 Dakikayı İyi Kullanmak



Puan başına ≈ 0,9 dakika: 2 puanlık bir soru 10 dakika sürmemeli. [Süreler 90 dakikalık bir sınav varsayar.]

Sınavdan önce

- 1–7. problemleri bakmadan yeniden çözün.
- Notlardaki deneme sınavını 90 dakikada yazın, sonra puanlayın.
- Kendi sayılarınızı denetlemek için code/*.py betiklerini çalıştırın.

Sınav sırasında

- En iyi bildiğinizle başlayın.
- Gantt şemalarını ve çerçeve tablolarını zamanlarıyla düzgünce çizin.
- Takıldınız mı? Formülü yazın ve geçin — sonra geri dönün.

Kısmi puan

- Yöntem sayılır: formül, yerine koyma, sonuç.
- Doğru yöntemle bulunan yanlış bir sayı da puan alır.
- Her yanıta birim yazın.

KISA SINAV

Kendinizi Sınayın

1. Bir süreç t = 3'te geliyor, 4 ms işlemci süresine ihtiyaç duyuyor ve t = 15'te bitiyor. Dönüş ve bekleme süreleri nedir?

2. 5. problemdeki sayfa tablosuyla (2. sayfa → 6. çerçeve) 0x2ABC mantıksal adresini çevirin.

3. t_{TLB} = 10 ns, t_m = 100 ns, isabet oranı α = 0,9. Etkin erişim süresi (ders uzlaşımı)?

4. ln -s a.txt b.txt, ardından rm a.txt. Veriyi b.txt üzerinden hâlâ okuyabilir misiniz? Neden?

8. Hafta: Akılda Kalması Gerekenler

- **1–4. haftalar:** tarihçe ve hedefler, ikili mod ve sistem çağrıları, süreç durumları ve PCB, iş parçacıkları — her birinin tanımını ve bir örneğini bilin.
- **Zamanlama:** Gantt şemasını çizin, sonra dönüş = tamamlanma − geliş ve bekleme = dönüş − patlama; uzlaşımları yazın.
- **Eşzamanlama:** counter++ = oku–ekle–yaz; üç gereksinim; muteks ve semaforlar; kilitlenmeye karşı kilit sırası.
- **Bellek:** taban + sınır denetimi, uyan stratejileri ve parçalanma, $p \mid d$ ayrımı, $\mathrm{EAT} = t_{TLB} + (2 - \alpha)\, t_m$.
- **Sanal bellek ve dosyalar:** $\mathrm{EAT} = (1 - p)\, t_m + p\, t_{fault}$; FIFO / OPT / LRU tabloları; blok $= \lfloor kR/B \rfloor$; sabit ve sembolik bağlantılar.
- **Alıştırma:** notlardaki deneme sınavı ve örnek kâğıt — saat tutarak.

GELECEK HAFTA

Linux Kabuğuna Giriş ve Dosya Sisteminde Gezinme

9. haftanın konuları:

- Linux edinmek: laboratuvar bilgisayarları, WSL (wsl --install) ya da VirtualBox'ta bir Ubuntu sanal makinesi
- Terminal, kabuk ve çekirdek — kabuk bir sistem programıdır (2. hafta)
- İlk komutlar: whoami, date, uname -a, pwd, ls, cd
- Gerçek dizin ağacında mutlak ve göreli yollar (7. hafta)
- Yardım almak: --help, man, sekme tamamlama, komut geçmişi

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz